

PRIMER DRAFT (22/1/26)

Rocio Perez Lopez

Lo más relevante de mis apuntes para SVR (y por qué nos sirve)

A) “Ya no hay línea: hay hiperplano” (SVR vive ahí)

Yo me quedo con una idea que se repite en clase: cuando tenemos varias variables, **no buscamos una recta**, buscamos una **superficie** en un espacio de muchas dimensiones (un **hiperplano**). Y justo ahí encaja SVR/SVM: el modelo no intenta “dibujar una recta bonita”, sino **ajustar una función con un margen**.

Cómo lo diría en la presentación/README:

“Con SVR no intentamos pasar por todos los puntos: intentamos capturar la tendencia manteniendo un margen de tolerancia.”

B) “No me fijo en la forma: me fijo en los resultados”

Otra cosa que me parece clave: **da igual si el modelo es lineal, polinómico, árbol o SVR**, lo que manda es:

- **métricas** (MAE / RMSE / R^2),
- y **generalización** (que no se dispare el overfitting).

Esto nos evita hacer “SVR porque sí”. Nosotros lo vamos a justificar con evidencia.

Traducción práctica:

- Primero mostramos un **baseline simple** (Linear/Ridge).
- Luego metemos **SVR**.
- Nos quedamos con SVR **si mejora MAE/RMSE** sin empeorar el overfitting.

C) “No siempre conviene hacer 3 splits” → con 4009 filas, mejor CV

En los apuntes aparece literalmente la idea: si el dataset no es enorme, separar en train/val/test puede dejarte con poco train; **cross-validation** suele ser mejor.

Aplicación directa:

- Nosotros deberíamos usar **K-Fold CV** para elegir hiperparámetros con **Optuna**.
- Y dejamos un **test final separado** solo para el informe final.

D) Regla de oro: “Si ya tienes test separado, no lo mezcles”

Esto también es importante: **no tocamos el test para ajustar**, porque si no, nos autoengañamos con las métricas.

Cómo lo contaría:

“El test es nuestra auditoría final. Optuna y decisiones se hacen con CV dentro de train.”

E) Del vídeo: SVM/SVR es potente pero exige tuning

En el vídeo se repite que SVR:

- suele rendir bien,
- pero requiere **ajuste fino** (hyperparams),
- y puede ser lento si el dataset crece mucho.

En nuestro caso:

- **4009 filas es un tamaño razonable** para SVR.
 - El riesgo real no es el tiempo, sino:
 - **preprocesado**,
 - y **tuning** (C, epsilon, gamma, kernel).
-

Lo que yo haría con SVR en nuestro dataset (con pensamiento crítico)

2.1 Preprocesado “mínimo pero con intención”

Como SVR depende de distancias (sobre todo con RBF), para mí esto es obligatorio:

- **Escalado de numéricas** sí o sí (StandardScaler o RobustScaler).
- Con categóricas: cuidado con un one-hot gigante, porque model tiene **1898 valores**.

Decisión crítica para no reventar dimensionalidad:

- brand: one-hot (57 → ok).
- fuel_type, transmission, accident: one-hot (pocas categorías → ok).
- model: yo **no** haría one-hot completo al principio. Mejor:

- **frequency encoding** (frecuencia del modelo), o
- agrupar modelos raros como “Other” si aparecen menos de N veces.

Esto lo podemos defender muy bien en la demo porque no es “copia-pegar”: es una decisión real por coste/beneficio.

2.2 Feature engineering barato pero rentable

Yo tocaría lo mínimo que suele dar mucho retorno:

- De engine (texto): sacar variables numéricas simples:
 - litros (2.0L), cilindros (8 Cylinder), horsepower (355.0HP), turbo sí/no.
 - Esto muchas veces mejora más que estar 2 horas afinando el kernel.
 - De milage: convertir "110,000 mi." a entero.
 - De price: probar entrenar con $\log_{10}(\text{price})$ para suavizar outliers, y luego invertir en la salida para mostrar precio real.
-

2.3 Optuna con objetivo claro y control de overfitting

Como nos piden overfitting < 5%, yo propondría medirlo así:

- $\text{gap} = (\text{RMSE}_{\text{val}} - \text{RMSE}_{\text{train}}) / \text{RMSE}_{\text{train}}$
- Si el gap se pasa, **penalizamos** la trial en Optuna.

Hiperparámetros mínimos a tunear:

- kernel: empezar con rbf vs linear.
 - C, epsilon
 - gamma (si usamos RBF)
-

Ideas

Idea 1) “SVR no tiene feature importance... pero nosotros lo resolvemos”

SVR no te da importancias directas como un árbol, así que:

- usamos **Permutation Importance** sobre el pipeline final,
- y lo complementamos con gráficos de:
 - **predicción vs real**

- **residuos** (para explicar errores y sesgos)

Esto cubre la parte de “explicación del rendimiento”

Idea 2) Dashboard como producto

Yo lo haría con 4 KPIs y 3 gráficos claros:

KPIs:

- Precio **medio** y **mediana** (para hablar de outliers)
- **RMSE** y **MAE** final (en test)
- (opcional) R^2

Gráficos:

- Top 10 marcas por **precio mediano**
 - model_year vs price (con tendencia)
 - milage vs price (con tendencia)
-

Idea 3) Streamlit con feedback real (nivel medio sin humo)

Esto nos sube de nivel sin infraestructura pesada:

- Usuario mete datos → predicción.
- Bloque opcional: “Si sabes el precio real, introdúcelo”.
- Guardamos en CSV o SQLite: inputs + predicción + real.
- Mostramos un mini panel con “error medio del feedback” a lo largo del tiempo.

Cumple “monitorización” y “recogida de nuevos datos” de forma realista.

Idea 4) Capa de “calidad del dato” antes de predecir

Para que el producto sea serio:

- validaciones: año razonable, mileage no negativo, etc.
- mensajes claros si falta algo

Esto reduce basura en feedback y nos da puntos por enfoque producto.

Idea 5) Encaje Agile que podemos usar en README

Yo lo organizaría así:

- Backlog priorizado (MVP primero: limpieza → baseline → SVR → demo).
- Kanban: Backlog → Ready → In progress → In review → Done.
- 3 sprints en 1 semana:
 1. EDA + limpieza + baseline + split/CV
 2. SVR + Optuna + evaluación + explicación (residuos/permutación)
 3. Streamlit + dashboard + README + presentación

-
- “Elegimos SVR porque, en datos tabulares con relaciones no lineales, el kernel RBF puede capturar complejidad sin imponer una forma fija.”
 - “Como el dataset no es enorme, priorizamos validación cruzada para estimar rendimiento con menos sesgo.”
 - “El test se mantiene aislado: lo usamos como auditoría final para evitar optimismo artificial.”
 - “SVR no trae importancias de serie; por eso usamos permutación y análisis de residuos para explicar el comportamiento del modelo.”

DATA SET INFO:

https://www.kaggle.com/datasets/taeefnajib/used-car-price-prediction-dataset?select=used_cars.csv